Final Report

Group No. 19

1. **Project Title**: Network-Aware Pod Placement in Kubernetes

2. **Team Members**: Vaibhav Lakhera (MT25049), Vishal (MT25053), Anshul (MT25062)

3. **Problem**

The Kubernetes scheduler places pods based on CPU and memory resources, as well as basic rules such as affinity and taints. It does not consider how services communicate with each other at runtime. In microservices, services often depend heavily on each other, and if they are placed on different nodes, each request has to cross the network, increasing latency.
The main issue is that scheduling is static and does not adapt to real-time traffic patterns. Our goal is to detect high communication between services and automatically move them closer (to the same node) without modifying application code or stopping the system.

4. **Motivation**

Microservices involve multiple internal calls for a single user request. Even small delays between services can increase total response time. Cross-node communication is slower due to network overhead and hardware delays.
Kubernetes does not adjust placement after deployment, so inefficient placements remain even if traffic patterns change. Solving this problem improves performance and reduces unnecessary network usage, which is important in real-world systems.

5. **Objectives**

The objective is to build a system that dynamically improves pod placement using runtime data. We designed a controller that reads service communication metrics from Linkerd and decides better placements. We aim to reduce latency, ensure zero downtime during movement, avoid unstable behaviour such as repeated shifting, and build a framework to clearly measure before-and-after performance.

6. **Background**

Kubernetes schedules pods only once during creation. To move a pod, its Deployment must be updated, which creates a new pod on another node.
Linkerd acts as a service mesh and provides metrics such as request rate and service-to-service latency. These metrics are stored in Prometheus and can be queried using PromQL.

We also use Linux tc netem to simulate network delay, helping us test how placement affects performance. Node affinity controls which node a pod is scheduled to.

7. **Methodology**

The system works as a feedback loop with three main components:

a) Microservices, b) Monitoring (Linkerd + Prometheus), and c) Custom Controller.

The microservices form a communication chain:
loadgen →gateway → auth-service → db-service, and gateway → user-service → db-service, along with calls to logger-service. This creates multiple service interactions, leading to latency buildup.
Linkerd is injected into every pod and captures all service-to-service communication. It provides metrics like request rate (RPS) and latency. Prometheus collects these metrics, and the controller queries them periodically.
The controller runs every 30 seconds and performs the following steps:

- It queries Prometheus to get the request rate and P95 latency for each service pair.
- It calculates a benefit score using:
  benefit = request_rate × latency
  This identifies the most costly communication edges.
- It sorts edges by this score and selects the highest-scoring one.
- Before moving a service, it checks:
  - whether the service is movable
  - whether the cooldown time has passed
  - whether it has already been moved (to avoid oscillation)
  - whether the source and destination are on different nodes
  - whether the target node has enough resources
- If all conditions are satisfied, it updates the Deployment with node affinity to move the service to the target node.

The movement happens using rolling updates, where a new pod is created first, and the old one is removed only after the new one is ready, ensuring zero downtime.
To maintain stability:

- Only one service is moved per cycle
- Each service is moved only once
- Cooldown prevents frequent updates

This approach allows the system to gradually improve placement and then stabilise without unnecessary changes.

8. **Implementation**

The system was implemented on a 3-node Kubernetes cluster using kubeadm.

One node as control plane (Master node) and other two nodes as Worker nodes.

All microservices were built using Python. Instead of separate implementations, a single reusable HTTP server was used and configured using environment variables such as service

name, upstream services, and processing delay. This simplifies development while maintaining realistic behavior.
A load generator continuously sends requests to the gateway at a fixed rate (~2.5 requests/sec). It logs response times and status codes for evaluation.
Linkerd was installed to provide service mesh functionality. It injects sidecar proxies into pods and exposes metrics. Prometheus collects these metrics and allows querying.

The controller was written in Python using the Kubernetes API. Its main tasks include:

- fetching metrics from Prometheus
- reading current pod placements and node resources
- computing benefit scores
- selecting services to move
- updating Deployment specifications

When moving a service, the controller:

- adds node affinity to target a specific node
- adds annotations to track movement and cooldown

This triggers a rolling update handled by Kubernetes.

To simulate network delay, tc netem was used to add 50ms latency on one worker node's network interface. This creates a clear difference between same-node and cross-node communication.
Automation scripts were created for building images, deploying services, running experiments, and collecting results. A data collection script records latency metrics, pod placements, and controller actions, and generates comparison plots.
The implementation focuses on simplicity, modular design, and reproducibility while demonstrating a realistic system.

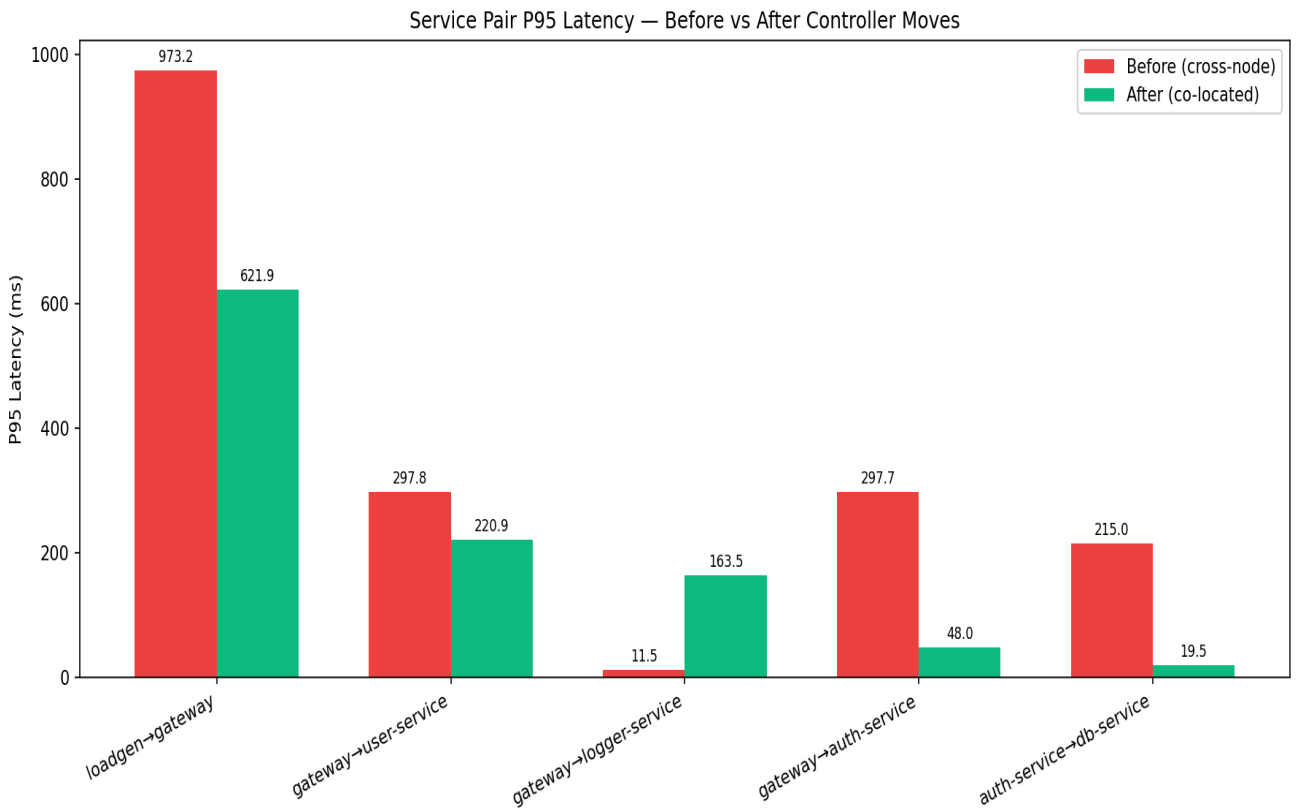
9. **Evaluation & Observations**

The experiment was conducted on a real 3-node cluster. Initially, all services were placed on one node and the database on another, creating multiple cross-node calls. A 50ms delay was added to simulate network latency.

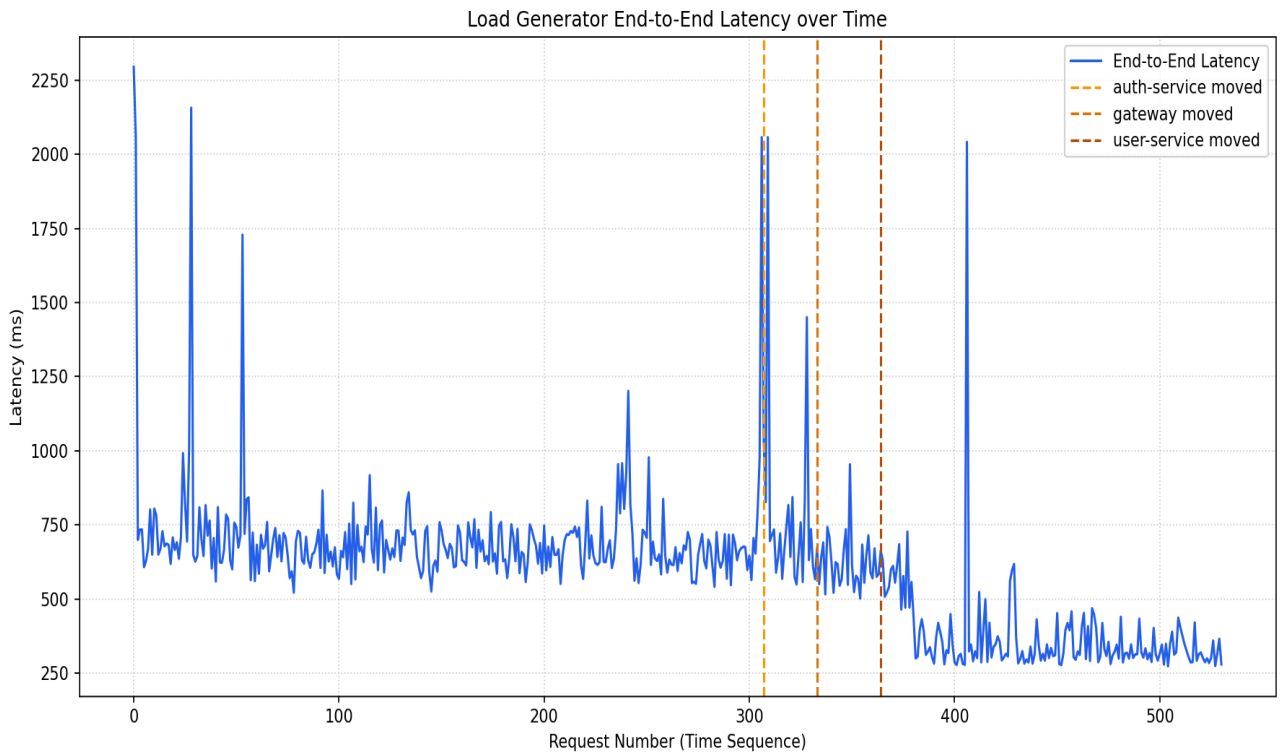
Baseline data was collected before starting the controller. After enabling the controller, services were gradually moved to better nodes.
Results showed:

- Average latency reduced from 685 ms to 447 ms (34.7% improvement)
- P95 latency improved by about 12%
- auth-service → db-service improved by ~91%

The controller stabilized after a few moves and did not oscillate. Some low-priority edges showed slight increase in latency, which is expected due to trade-offs.



Latency: Before vs After



Latency over Time

10. **Outcomes & Conclusion**

The project successfully demonstrates that runtime network-aware pod placement is possible using existing Kubernetes features. It significantly reduces latency without modifying application code or replacing the scheduler.
The system maintains availability during updates and converges to a stable state. The approach is simple, effective, and practical for real-world microservice environments.
Overall, the project shows how runtime telemetry can be used to improve system performance dynamically.

11. **References**

> https://kubernetes.io/docs/home/
> https://prometheus.io/docs/prometheus/latest/getting_started/
> https://linkerd.io/2-edge/overview/
> https://man7.org/linux/man-pages/man8/tc-netem.8.html

Github Link:

https://github.com/vaibhav-init/Network-Aware-Pod-Placement-in-Kubernetes